

Algorithms Portfolio

Curated algorithmic solutions with correctness and complexity analysis

Ryan McMillan

MCompSci (cand.) – University of New England

C++ primary • Python secondary

January 18, 2026

License

This work is licensed under the MIT License. The canonical source and latest version are available at: <https://github.com/rmcmillan34/algorithms-portfolio>.

Contents

1	Introduction	3
1.1	How to Use This Book	3
2	Arrays & Sequences	4
2.1	Core Ideas	4
2.2	Common Pitfalls	4
2.3	Easy	4
2.3.1	Plus One	4

1 Introduction

This document is a curated portfolio of algorithmic solutions. Each selected problem includes:

- clear problem restatements,
- key invariants / observations,
- pseudocode,
- correctness sketches,
- complexity analysis,
- C++ (primary) and Python (secondary) implementations.

The focus is on clarity, correctness, and reasoning.

1.1 How to Use This Book

Problems include an optional **Author's Thinking** block, the authors plain language thinking and problem framing, which is excluded from exemplar builds.

Use this book to understand how to formally reason with and structure algorithm and data structures problems.

2 Arrays & Sequences

Arrays are the foundational data structure in most algorithmic problems. They provide constant-time indexed access and strong cache locality, but require careful reasoning about indices, bounds, and in-place mutation.

This chapter focuses on recognizing array-based problem patterns and maintaining clear invariants over contiguous memory.

2.1 Core Ideas

- Index-as-state reasoning
- In-place mutation vs auxiliary storage
- Prefix and suffix accumulation
- Simulation of elementary operations (e.g. addition, carry propagation)

2.2 Common Pitfalls

- Off-by-one errors at boundaries
- Unintended data overwrites during in-place updates
- Assuming sorting or uniqueness without verifying constraints
- Over-allocating space when a single pass suffices

2.3 Easy

The following problems introduce fundamental array manipulation techniques. The emphasis is on establishing and maintaining simple invariants while iterating through the array.

2.3.1 Plus One

Difficulty: Easy

Tags: Arrays, Simulation

ID: LC-066

Problem. Given a non-empty array of digits representing a non-negative integer, increment the integer by one and return the resulting array of digits.

The most significant digit is stored at the beginning of the array, and each digit is in the range $[0, 9]$.

Key idea. Incrementing the number requires simulating elementary addition from right to left. Carry propagation only affects trailing digits equal to nine; once a digit strictly less than nine is incremented, the process terminates.

Author's Thinking.

My initial instinct was to convert the digit array into an integer, add one, and convert it back. This approach fails due to potential overflow and violates the problem constraints.

What simplified the reasoning was treating the operation as grade-school addition and maintaining the invariant that all digits to the right of the current index are already correct after carry handling.

Algorithm.

```

1 for i from n - 1 downto 0:
2     if digits[i] < 9:
3         digits[i] = digits[i] + 1
4         return digits
5     digits[i] = 0
6
7 prepend 1 to digits
8 return digits

```

Listing 1: Pseudocode

Correctness sketch. We iterate from the least significant digit toward the most significant digit. At each iteration, the invariant maintained is that all digits to the right of the current index represent the correct result after increment and carry propagation.

If a digit less than nine is encountered, incrementing it produces the correct result and no further carry is required. If all digits are nine, each is set to zero and a leading one is prepended, yielding the correct incremented number. Thus, the algorithm always returns the correct result.

Complexity. The algorithm runs in $O(n)$ time, where n is the number of digits. It uses $O(1)$ additional space beyond the output array.

C++ Implementation

```

1  /*
2  * LC-066 - Plus One
3  *
4  */
5
6  #include <iostream>
7
8  class Solution {
9  public:
10     vector<int> plusOne(vector<int>& digits) {
11         for(int i = digits.size(); i-- > 0 ; ) {
12             if (digits[i] != 9) {
13                 digits[i]++;
14                 return digits;
15             }
16             else {
17                 digits[i] = 0;
18             }
19         }
20
21         vector<int> newDigits(digits.size() + 1);
22         newDigits[0] = 1;
23         for (int j = 1; j < newDigits.size(); j++) {
24             newDigits[j] = digits[j-1];
25         }
26
27         return newDigits;
28     }
29 };

```

Listing 2: C++ Implementation

Python Implementation

```

1  # LC-066 Plus One
2

```

```
3 from typing import List
4
5 def plusOne(digits: List[int]) -> List[int]:
6     for i in range(len(digits) - 1, -1, -1):
7         if digits[i] < 9:
8             digits[i] += 1
9             return digits
10        digits[i] = 0
11
12    return [1] + digits
```

Listing 3: Python Implementation
